

UNIDAD 3 - DOCKER: LA RECETA REPRODUCIBLE

Taller de aula: leer una receta que todavia no corriste

Programacion 3 - Tecnicatura Universitaria en Programacion - UTN FRM

Modalidad	En grupos de dos o tres, en el aula. Se entrega antes de irse : lo que no se termina en la mesa no se termina.
Requisitos	No hace falta tener Docker instalado. Tampoco hace falta internet. Solo el apunte del capitulo 3 a mano.
Tiempo	70 minutos. Partes 1 a 5, mas la Parte 6 que se lleva al proyecto.
Puntaje	100 puntos: 1 (20) + 2 (15) + 3 (20) + 4 (15) + 5 (20) + 6 (10).
Que se evalua	Criterio de construccion: decidir que entra a la imagen, en que orden y por que, y poder justificar cada linea con el mecanismo que la explica.
Integrantes	

Antes de arrancar

Este taller no pregunta que instrucciones recordas: pregunta que **decidirias** al escribir una receta. Todas las salidas de comandos que vas a leer estan impresas aca; no hay nada que construir ni que ejecutar. Un `Dockerfile` se lee como se lee un programa: cada linea tiene un efecto, un costo y un momento. Y la regla de oro sigue siendo la de la clase pasada: **una decision sin justificacion no suma puntaje**. "Porque en el tutorial estaba asi" no es una razon de ingenieria.

Parte 1 - Inventario de instrucciones: que pesa y que solo anota

20 puntos

Este es el archivo de construccion del backend del proyecto, sin los comentarios y con las lineas numeradas. Es la misma receta que Easypanel va a ejecutar en el servidor en la Clase 4.

```
# backend/Dockerfile
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY main.py .
6 COPY db.py .
7 EXPOSE 8000
8 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

1.1 Completa el inventario

12 puntos (2 por fila)

Una fila por instruccion. En "Que la invalida" escribi el cambio mas chico que obliga a volver a ejecutarla. En la ultima columna, que pasaria si borraras esa linea del archivo.

Linea	Capa o metadato?	Que hace	Que la invalida (el cambio mas chico)	Si la borro del archivo...
1				
2				
3 y 4				
5 y 6				
7				
8				

1.2 Predecir la salida de `docker history`

4 puntos

Sin correr nada: de las ocho lineas, cuantas van a aparecer con un tamaño distinto de 0B? Nombralas y ordenalas de mayor a menor peso esperado, con una línea de justificación para la primera.

1.3 La instrucción que no está

4 puntos

El archivo real del proyecto copia `main.py` y `db.py` por separado, en dos líneas, pudiendo escribir `COPY . .` en una sola. Da las **dos** razones por las que se eligió la versión larga. Una es de rendimiento y la otra es de otra categoría: decime cuál es cuál.

Parte 2 - La cascada del cache

15 puntos

Un grupo construyo la imagen del backend y despues hizo tres cambios, uno por vez, reconstruyendo despues de cada uno. Estas son las tres salidas, recortadas a lo que importa. Nadie toco requirements.txt salvo donde se indica.

```
--- Construccion A: se cambio una linea de codigo en main.py ---
=> [1/6] FROM python:3.12-slim          CACHED
=> [2/6] WORKDIR /app                    CACHED
=> [3/6] COPY requirements.txt .          CACHED
=> [4/6] RUN pip install --no-cache-dir -r requirements CACHED
=> [5/6] COPY main.py .                  0.1s
=> [6/6] COPY db.py .                    0.1s
tiempo total: 1.9s

--- Construccion B: se agrego una dependencia a requirements.txt ---
tiempo total: 118.4s

--- Construccion C: se renombro WORKDIR /app a WORKDIR /aplicacion ---
tiempo total: 121.7s
```

2.1 Traza las tres construcciones

8 puntos

Complete la tabla marcando CACHED o EJECUTA en cada celda, y marca con un circulo, en las columnas B y C, la instruccion exacta donde se rompio la cadena.

#	Instruccion	A: cambio main.py	B: cambio requirements.txt	C: renombro WORKDIR
1	FROM python:3.12-slim			
2	WORKDIR			
3	COPY requirements.txt .			
4	RUN pip install ...			
5	COPY main.py .			
6	COPY db.py .			

2.2 Explicalo en una frase de ingenieria

4 puntos

Completa: "Renombrar el directorio de trabajo volvio a descargar todas las dependencias porque la clave del cache de una instruccion es _____ mas _____, y porque cuando una capa se invalida _____."

.....

.....

.....

2.3 El comando que muestra la verdad de adentro*3 puntos*

Nombra el comando que permite leer, capa por capa, que instruccion creo cada una y cuanto pesa. Y explica que cosa muestra ese comando que **ninguna** lectura del Dockerfile puede mostrarte.

Parte 3 - El orden de la receta**20 puntos**

Doce pasos, desordenados, para construir la imagen de una API en Python que necesita una biblioteca del sistema y que no debe correr como administrador. Once son instrucciones del Dockerfile; **uno no lo es**, y donde va ese es la mitad del ejercicio. El orden aca no es cuestion de estilo: hay posiciones que rompen la construccion, posiciones que rompen el despliegue de todos los dias y una que quema una credencial para siempre.

- a) COPY main.py .
- b) RUN pip install --no-cache-dir -r requirements.txt
- c) FROM python:3.12-slim
- d) USER aplicacion
- e) COPY requirements.txt .
- f) WORKDIR /app
- g) CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
- h) RUN apt-get update && apt-get install -y --no-install-recommends \libpq5 && rm -rf /var/lib/apt/lists/*
- i) EXPOSE 8000
- j) RUN useradd --create-home aplicacion
- k) escribir el archivo .dockerignore con .git/, .venv/, __pycache__/ y .env
- l) ENV PYTHONUNBUFFERED=1

3.1 Ordena los doce pasos**10 puntos**

Escribi las doce letras en el orden correcto de ejecucion.

3.2 Marca los tres puntos de no retorno**6 puntos (2 por punto)**

Tres de esos pasos, puestos fuera de lugar, hacen dano de tres clases distintas: uno rompe la construccion en el acto, uno no rompe nada y te cuesta minutos en cada despliegue, y uno no se puede deshacer reescribiendo el archivo. Identificalos y decime, en cada caso, que error concreto los convierte en trampa.

3.3 Los que no pesan y aun asi importan*4 puntos*

Tres de los once pasos que si son instrucciones no crean ninguna capa. Identificalos y explica por que su **posicion** en el archivo igual importa, si no pesan nada.

Parte 4 - Auditoria de un archivo de construccion

15 puntos

Un grupo entrego este Dockerfile como "listo para desplegar". Tiene **cinco problemas reales** -uno de ellos con dos defectos distintos en la misma linea- y **una linea que parece un problema y no lo es**.

```
# Dockerfile entregado por un grupo
1  FROM python:latest
2  WORKDIR /app
3  COPY . .
4  RUN pip install -r requirements.txt
5  RUN apt-get update
6  RUN apt-get install -y curl vim git
7  ENV DATABASE_URL=postgres://postgres:tup2026@db:5432/tup
8  EXPOSE 8000
9  CMD uvicorn main:app --host 127.0.0.1 --port 8000
```

4.1 Los cinco problemas

10 puntos (2 por problema)

Para cada uno: numero de linea, correccion concreta y una linea de justificacion. Sin justificacion no suma.

4.2 La linea que no es un problema

5 puntos

Un integrante lee en el apunte que EXPOSE no abre ningun puerto y propone borrar la linea 8, "porque no hace nada". Decime si tiene razon en el hecho y por que igual la decision es incorrecta. Nombra ademas quien consume esa linea.

Parte 5 - Peritaje: el contenedor que arranca y no anda**20 puntos**

Un grupo pide ayuda: "construimos las dos imagenes, los contenedores estan arriba, el log no tira ni un error y la calculadora no calcula". Un rato despues escriben de nuevo: "ah, y nos avisaron que la contrasena de nuestra base aparece publicada". Esta es toda la evidencia que trajeron. Leela entera antes de escribir nada.

```
$ docker ps
CONTAINER ID   IMAGE      STATUS          PORTS                               NAMES
a41c9f2b7e30   tup-api    Up 12 minutes   0.0.0.0:8000->8000/tcp              tup-api
7d2e5a1c9004   tup-web    Up 12 minutes   0.0.0.0:8080->80/tcp                tup-web

$ docker logs tup-api | tail -3
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

$ curl http://localhost:8000/api/salud
curl: (52) Empty reply from server

$ docker logs tup-web | tail -2
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/28 03:02:11 [notice] 1#1: start worker processes

$ docker exec tup-api ls -a /app
.  ..  main.py  db.py  requirements.txt

$ docker history tup-api
CREATED BY                                     SIZE
CMD ["sh" "-c" "uvicorn main:app --host 127.0..."]   0B
RUN /bin/sh -c rm -f .env                          0B
COPY . . # buildkit                                48.2MB
RUN /bin/sh -c pip install -r requirements.txt       61.4MB
WORKDIR /app                                         0B
FROM docker.io/library/python:latest                997MB

$ docker save tup-api -o api.tar && tar -xf api.tar
$ grep -r "DATABASE_URL" blobs/
blobs/sha256/9f3c1e...: DATABASE_URL=postgres://postgres:tup2026@db:5432/tup

$ time docker stop tup-api
tup-api
real    0m10.412s
```

5.1 Reconstrui la cadena de hechos

6 puntos

En seis lineas como maximo, contame que paso, en orden, desde que el grupo escribio el `Dockerfile` hasta que la contrasena termino publicada. Sostene cada afirmacion en una linea concreta de la evidencia.

5.2 Las tres contradicciones aparentes

6 puntos (2 cada una)

La evidencia parece contradecirse tres veces. Explica cada caso: (i) el log dice `Uvicorn running` y no hay un solo error, pero `curl` no obtiene respuesta; (ii) `docker exec ls -a` no muestra ningun `.env` adentro del contenedor, y sin embargo la credencial se extrae de la imagen; (iii) el contenedor del frontend arranco sin errores y la calculadora igual no funciona.

5.3 Los principios violados

4 puntos

Nombra los principios del capitulo que el grupo violo y en que decision concreta. Uno de ellos ya lo viste en la Clase 2 y aparece aca aplicado adentro de la imagen: decime cual y como se traduce.

5.4 Las cuatro correcciones, en orden*4 puntos*

Escribi las cuatro medidas que aplicarias, en el orden en que las aplicarias. Justifica el **orden**, no solo el contenido: una de las cuatro no puede esperar a las otras tres.

Parte 6 - El plan de imagenes de tu propio proyecto**10 puntos**

Esta parte no es un ejercicio de laboratorio: es el plano de lo que Easypanel va a construir en las Clases 4 y 5. Retoma la tabla de puertos de la Parte 6 del taller anterior, un nivel mas adentro: alla decidiste que se publica; aca decidis **que entra a cada imagen, en que orden y que se queda afuera**.

Servicio	Imagen base con etiqueta exacta	Que se copia y en que orden	Que NO entra (.dockerignore)	Que decide el entorno	Puerto interno
Frontend					
API					
Base de datos					

6.1 La decision que arrastra a todas las demas**5 puntos**

De todo lo que escribiste en tu tabla, una sola celda por fila condiciona a las otras cinco: si la cambias mas adelante, tenes que revisar la fila entera. Nombrala, decime por que, y explica que problema concreto te evita fijarla hoy con su etiqueta exacta.

6.2 La linea que te vas a agradecer**5 puntos**

Mira tu tabla y responde: si manana alguien de tu grupo dice "metamos el .env adentro de la imagen asi no hay que configurar nada en Easypanel y desplegamos mas rapido", que le decis? Escribi la respuesta tecnica **y la alternativa concreta** que le ofreces, porque la necesidad es legitima.

Cierre - Para discutir en la mesa**sin puntaje**

La conclusion del capitulo es una sola frase: **la receta es la misma en tu maquina y en el servidor**. No parecida, no equivalente, no "casi igual salvo un par de cosas". La misma. Desde el momento en que el entorno se escribe en un archivo versionado, "en mi maquina andaba" deja de ser una explicacion: la maquina paso a ser parte del codigo, y por lo tanto es revisable, comparable y discutible como cualquier otra linea que hayan escrito.

Discutan cinco minutos, sin escribir: en los proyectos que ya hicieron en esta carrera, cuanto de lo que hacia falta para que anduvieran estaba escrito en alguna parte, y cuanto vivia en la cabeza de un integrante? La version del interprete. Esa biblioteca del sistema que hubo que instalar a mano. El archivo de configuracion que se pasaron por mensaje privado. La base que alguien creo una tarde y nadie documento. Para cada una de esas cosas: en que linea de un `Dockerfile` iria, y que se pierde mientras no este escrita en ningun lado?

Lo que se entrega

Las hojas del taller, con el nombre de los tres integrantes en la primera. Si la Parte 6 les quedo a medias, entreguenla igual como esta: esa tabla es el plano de lo que Easypanel va a construir en la Clase 4, y la vamos a retomar con el proyecto delante. Lo que **no** se entrega en blanco es la justificacion: prefiero una decision discutible bien fundamentada que una correcta sin razon.